BTS CIEL - Informatique et Réseaux

Epreuve E6

Rapport technique MP1 - tâche 1



Étudiant :
RICARD Mathis
ROUMEC Clément

Établissement :
Enseignement Supérieur Carnus

Sommaire

Sommaire **2**
1. Présentation de la tâche **3**
1.1. Objectifs et périmètre fonctionnel 3
1.2. Contraintes techniques et environnement de développement 3
1.3. Interfaces avec les autres tâches du Mini-Projet 1 4
2. Analyse et conception **4**
2.1. Analyse du besoin et de la problématique 4
2.2. Étude comparative et justification des choix d'architecture 4
2.3. Architecture globale du système vidéo 5
3. Réalisation et mise en œuvre **6**
3.1. Déploiement matériel et alimentation 12 V DC 6
3.2. Configuration et intégration réseau sous Windows 6
3.3. Découverte réseau et outils Hikvision (HiTools Delivery) 6
3.4. Développement Python : scripts test_camera.py et server_ipcam.py 7
3.5. Interface Web de visualisation temps réel (camera.html) 7
4. Tests, recette et validation **8**
4.1. Tests fonctionnels 8
4.2. Procédures de test : 8
5. Résultats et limites **9**
5.1. Performances et résultats obtenus 9
5.2. Limitations identifiées 10
6. Difficultés rencontrées et solutions apportées **10**
6.1. Tableau récapitulatif des problèmes rencontrés 10
6.2. Analyse détaillée de l'incident majeur : conflit de port WebSocket 8585 10
7. Conclusion et perspectives **12**
7.1. Bilan du travail réalisé 12
7.2. Perspectives d'évolution 12
8. Annexes techniques **12**
8.1. Interface web 12

1. Présentation de la tâche

1.1. Objectifs et périmètre fonctionnel

Dans le cadre de notre Mini-Projet 1, la Tâche 1 qui nous a été confiée portait sur le choix, la mise en œuvre, l'intégration et la validation d'une solution de vidéosurveillance pour la salle 215, ainsi que d'autres fonctionnalités qui étaient attribués aux autres "sous-groupe" du mini-projet 1, c'est à dire Brassat Liam, Grimal Maxence et Mignet Naël. La tâche englobe :

- Le choix de la solution vidéo entre :
  OPTION A, avec une caméra d'ordinateur portable.
  OPTION B, avec une caméra professionnelle Hikvision.

- L'acquisition du flux vidéo de la caméra, et son traitement d'image.

- Le choix des données à journaliser et à envoyer.

- La détection d'évènements depuis le flux vidéo.

- La capture d'image (comme une preuve) dès qu'il y a du mouvement.

- La transmission de l'image captée au serveur, et le chemin de l'image à la base de données.

- Enfin les documents techniques de la tâche, et les tests.

1.2. Contraintes techniques et environnement de développement

La mise en œuvre a dû respecter plusieurs contraintes matérielles, réseau et logicielles :

- **Alimentation matérielle :** La caméra dôme Hikvision nécessitant une alimentation stabilisée 12 V DC avec un type de chargeur spécifique. Mathis à voulu la prendre chez lui pour avancer sur la caméra mais il n'avait pas de chargeur adapté, donc il n'a pas pu développer chez lui.
- **Isolement réseau local :** Une contrainte à été d'avoir le pc et la caméra sur le même sous-réseau pendant le test de la caméra car il pouvait y avoir un conflit d'adressage car le wifi et la caméra avaient la même IP.
- **Environnement de développement :** Le développement s'est fait avec Visual Studio Code car on était alaise avec, et python pour la librairie OpenCV recommandé par le professeur.

1.3. Interfaces avec les autres tâches du Mini-Projet 1

La Tâche 1 s'insère directement dans l'architecture globale du MP1 et interagit avec :

- **Tâche 2 (Détection de mouvement par capteur PIR par MIGNET Naël / GRIMAL Maxence) :** Leur objectif était de mettre en place un détecteur de mouvement. Cette tâche est complémentaire à la vidéo surveillance car elle permettait de détecter plus précisément des mouvements, tandis que la caméra elle servait de détection plus complémentaire mais offrant la possibilité de faire une capture d'écran de l'évènement.

- **Tâche 3 (Infrastructure réseau et serveur Linux - Brassat Liam) :** La Tâche 1 transmet les images capturées vers le serveur pour un stockage centralisé dans un répertoire dédié (accessible depuis internet). Les informations relatives aux événements (heure, nom du fichier image, chemin vers l'image) sont transmises à l'API REST de la Tâche 3 pour l'enregistrer dans la base de données.

2. Analyse et conception

2.1. Analyse du besoin et de la problématique

Le cahier des charges impose une surveillance automatisée capable de détecter toute présence physique dans la salle 215, de capturer une preuve visuelle de qualité sans engorger le stockage du serveur, et d'offrir à l'utilisateur un retour visuel temps réel via un navigateur web. La problématique centrale repose sur la conception d'un logiciel léger et robuste capable d'analyser jusqu'à 30 images par seconde, d'éliminer les fausses alertes dues au bruit numérique de la caméra, et d'assurer une reconnexion automatique en cas d'interruption inhabituelle.

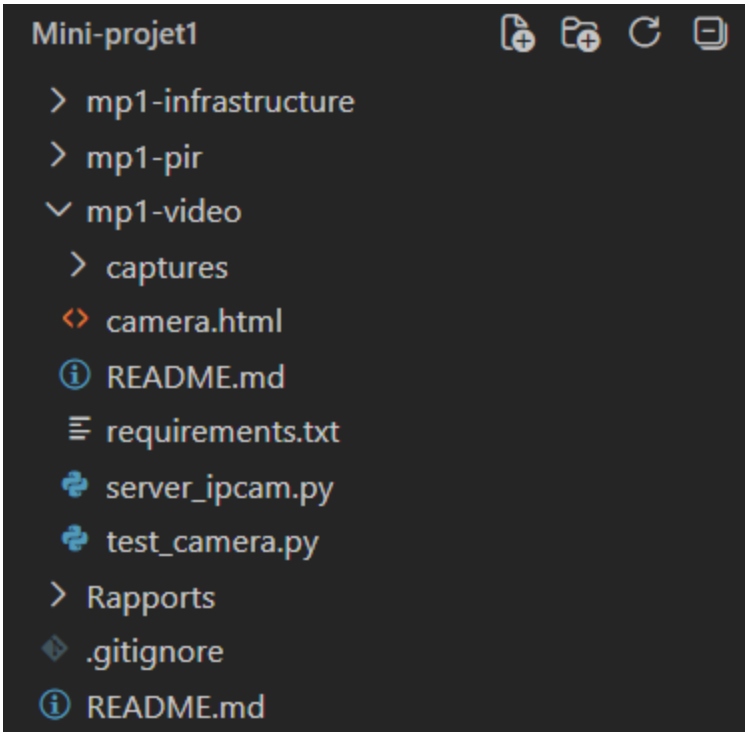
2.2. Étude comparative et justification des choix d'architecture

Le sujet du projet demandait de choisir et de justifier l'architecture matérielle d'acquisition :

Solutions	**Solution A : PC Portable + Caméra intégrée**	**Solution B : Caméra Hikvision**
Aisance avec matériel	Plutôt aisé, car même si c'est quelque chose qui n'a jamais été vu c'est plus facile à mettre en oeuvre qu'une vrai caméra complexe	Difficile car les caméra Hikvision nécessitent plusieurs prérequis pour l'installation avant d'avoir l'accès à la caméra.
Simplicité de mise en œuvre	Assez compliqué	Très compliqué
Qualité de la caméra	Basique : Caméra de PC, grosse qualité, pas forcément adapté à la vidéo surveillance	Industrielle : Caméra dôme, peu coûteuse car elle spécialisé pour avoir une qualité minimale pour pas consommer trop d'espace
Coût et disponibilité	Moins coûteux que la caméra dôme	Plus coûteux que la caméra PC

Justification du choix : La Solution B avec caméra IP Hikvision et PC d'acquisition a été retenue pour sa robustesse matérielle, la qualité du flux vidéo et son potentiel avec le projet.

2.3. Architecture globale du système vidéo



Le système de vidéo surveillance est placé dans le dossier "mp1-vidéo" du Mini projet.

3. Réalisation et mise en œuvre

3.1. Déploiement matériel et alimentation 12 V DC

Le déploiement physique a débuté par le raccordement de la caméra IP Hikvision DS-2CD2112F-I :

- Vérification du bloc d'alimentation externe : sortie 12 V ⎓ (courant continu DC), intensité de 1 A.
- Branchement de la prise jack sur le connecteur d'alimentation de la caméra.
- Connexion de la prise RJ45 du câble Ethernet sur le port carte réseau du PC .

3.2. Configuration et intégration réseau sous Windows

L'interface réseau Ethernet du PC se trouvant initialement en adresse automatique (169.254.182.94) sans serveur DHCP, une IP statique a été configurée dans le même sous-réseau que la caméra :

-> Exécution dans PowerShell Administrateur
netsh interface ipv4 set address name="Ethernet" static 192.168.1.10 255.255.255.0

-> Validation de la connectivité ICMP avec la caméra
ping 192.168.1.12
ping valide

Le ping permettait de vérifier si le pc arrivait à communiquer avec la caméra après le changement IP.

3.3. Découverte réseau et outils Hikvision (HiTools Delivery)

Suite à une difficulté lors de l'exécution de l'installateur Hikvision SADP, l'outil moderne **HiTools Delivery** a été déployé . Il a permis d'identifier immédiatement la caméra sur le sous-réseau avec les paramètres d'usine suivants :

- Modèle : DS-2CD2112F-I
- Adresse IPv4 : 192.168.1.12
- Port RTSP : 554
- Identifiants par défaut : admin / 12345

3.4. Développement Python : scripts test_camera.py et server_ipcam.py

Le développement logiciel s'est appuyé sur ces deux scripts principaux :

1 - test_camera.py : Script d'analyse simplifiée permettant de valider la capture OpenCV sur l'URL RTSP rtsp://admin:12345@192.168.1.12:554/Streaming/Channels/102 et d'afficher le nombre d'images par seconde.
2 - server_ipcam.py : Application serveur complète structurée comme suit :

- **Algorithme de détection :** Conversion de chaque frame en niveaux de gris, application d'un floutage Gaussien, calcul de la différence par rapport à l'image de référence (cv2.absdiff), seuillage binaire (cv2.threshold à 25) et comptage des pixels non nuls via la fonction cv2.countNonZero(thresh). Toutes ces fonctions viennent de la librairie OpenCV et on été développées avec l'aide d'une IA car ce type de code n'avait jamais été vu auparavant.
- **Gestion du seuil adaptatif :** Seuil par défaut fixé à 47 pixels, modifiable en direct sur le site internet via WebSocket. (http://127.0.0.1:8000/camera.html)
- **Enregistrement des captures :** Lors d'une détection (durant au moins 4 trames consécutives), incrustation de l'horodatage courant et sauvegarde automatique au format JPEG dans le répertoire captures/ avec un intervalle de sécurité de 15 secondes entre chaque photo pour limiter au maximum la saturation du stockage.

3.5. Interface Web de visualisation temps réel (camera.html)

Une interface utilisateur moderne a été créée en HTML, CSS et JavaScript, aux couleurs du site de Carnus Rodez. Ses fonctionnalités comprennent :

- Affichage du flux vidéo en direct.
- Bascule instantanée entre la vue Image couleur (avec encadrement des zones mobiles) et la vue Masque binaire d'OpenCV.
- Graphique d'activité temps réel représentant le pourcentage de pixels modifiés sur les 60 dernières secondes par rapport au seuil.
- Curseur logarithmique de réglage du seuil de sensibilité (de 1 à 100 000 pixels, plus le seuil est haut, plus il faut de mouvement pour détecter un événement).
- Commandes d'arrêt/reprise de la détection, déclenchement manuel de photo, activation d'alerte sonore et mode plein écran.
- Galerie interactive avec consultation des clichés capturés en modal (``).

4. Tests, recette et validation

4.1. Tests fonctionnels

ID Test	Objectif	Conditions & Prérequis	Procédure / Actions	Résultat attendu	Résultat obtenu	Statut
T1-01	Validation connectivité réseau IP	Câble RJ45 branché, PC en 192.168.1.10	ping 192.168.1.12 dans PowerShell	4 réponses obtenues, temps < 5 ms	4/4 réponses, temps moyen 1 ms	**OK**
T1-02	Test vidéo caméra avec VLC	VLC installé	Ouvrir l'URL RTSP Channel 101/102 dans VLC	Affichage du retour vidéo	Retour vidéo en direct	**OK**
T1-03	Exécution script de test OpenCV	Python 3 & OpenCV installés	Exécuter python test_camera.py	Fenêtre d'affichage à 352x288 / 640x480 pixels	Affichage OK, fermeture sur touche 'q'	**OK**
T1-04	Affichage de l'interface web de vidéosurveillance	Ports 8000 et 8585 libres	Exécuter python server_ipcam.py	Démarrage des services HTTP et WS sans erreur	Services démarrés, navigateur ouvert automatiquement	**OK**
T1-05	Détection de mouvement & Captures	server_ipcam.py en cours d'exécution	Provoquer un mouvement dans le champ visuel de la caméra	Encadrement des zones en mouvement, alerte, fichier .jpg généré dans captures/	Capture enregistrée avec timestamp	**OK**

4.2. Procédures de test :

Voici la liste détaillé des procédures de tests pour vérifier l'état de marche du système :

Test, retour vidéo caméra :

- Installer VLC : https://www.videolan.org/vlc/index.fr.html
- Brancher la caméra Hikvision à votre PC via un câble ethernet.
- Brancher la caméra Hikvision sur secteur avec une alimentation 12V.
- Ouvrir VLC et aller dans l'onglet "MEDIA".
- Cliquez sur "Ouvrir un flux réseau…"

- Dans l'onglet "Réseaux" collez cette url : rtsp://admin:12345@192.168.1.12:554/Streaming/Channels/102
- Validez en cliquant sur "Lire"

Vous devriez apercevoir la vidéo de la caméra. Si cela est réalisé, la caméra est fonctionnelle est bien reliée au PC. Sinon re-tentez les étapes dans l'ordre.

Test, retour vidéo caméra via le code :

- Téléchargez le projet en entier sur : https://github.com/mathisricard-maker/Mini-Projet1/archive/refs/heads/Tests.zip
- Brancher la caméra Hikvision à votre PC via un câble ethernet.
- Brancher la caméra Hikvision sur secteur avec une alimentation 12V.
- Lancer le programme test_camera.py dans *Mini-projet1/mp1-video/test_camera.py*

Si vous apercevez le retour vidéo c'est que la caméra est bien fonctionnelle, reliée au pc et que le code fonctionne. Sinon re-tentez les étapes dans l'ordre.

Test, fonctionnement du site web, détection de mouvement et capture d'image :

- Téléchargez le projet en entier sur : https://github.com/mathisricard-maker/Mini-Projet1/archive/refs/heads/Tests.zip
- Brancher la caméra Hikvision à votre PC via un câble ethernet.
- Brancher la caméra Hikvision sur secteur avec une alimentation 12V.
- Lancer le programme python "server_ipcam.py" dans *Mini-projet1/mp1-video/server_ipcam.py*

Si la page s'ouvre passez aux étapes suivantes sinon recommencez dans l'ordre.

- Cliquez sur les boutons "Mettre en pause" puis "Image" puis "Masque".

Si la vidéo se met en pause / s'affiche correctement / se masque. Alors ces boutons fonctionnent.

- Passez votre main ou quelconque objet devant la caméra pour créer du mouvement.

Si la caméra détecte le mouvement sur le retour vidéo et prend une capture d'écran qui s'affiche plus bas, alors cette fonctionnalité marche. Si toutes ces étapes sont correctement réalisées, alors le système de vidéosurveillance (uniquement) est entièrement fonctionnel.

5. Résultats et limites

5.1. Performances et résultats obtenus

- **Fluidité et réactivité :** Le décodage du flux secondaire RTSP (Channel 102) couplé à la transmission WebSocket binaire permet d'obtenir un affichage à 25-30 images par seconde parfaitement.
- **Fiabilité de la détection :** L'association du flou Gaussien et du filtrage par surface de contour élimine la totalité du bruit capté en faible luminosité .
- **Interface Homme Machine (IHM) :** L'interface camera.html offre une ergonomie intégrant le suivi d'activité et le contrôle dynamique des seuils sans interruption de la vidéo.

5.2. Limitations identifiées

- **Sensibilité aux variations d'éclairage global :** L'allumage ou l'extinction soudaine des néons de la salle 215 provoque une modification globale des pixels qui déclenche une capture ponctuelle, pourtant il ne s'agit pas vraiment d'un mouvement humain.
- **Reconnaissance :** L'algorithme basé sur la différence d'images ne distingue pas un être humain d'un animal ou d'un objet en mouvement, ce qui pourrait éviter les captures inutiles si par hasard un animal venait à passer devant la caméra par exemple.

6. Difficultés rencontrées et solutions apportées

6.1. Tableau récapitulatif des problèmes rencontrés

Problème rencontré	Analyse de la cause	Solution corrective appliquée
Erreur « Error launching installation » lors du lancement de SADP	Exécution du fichier d'installation directement depuis l'archive compressée .zip sans extraction .	Installation de HiTools Delivery qui est un complément du SADP et qui fonctionne tout aussi bien
Recherche du mode d'emploi de la caméra	L'absence complet de guide d'utilisation à fait qu'il a fallu faire de l'OSINT pour trouver comment la faire fonctionner	Il faut aller sur le site d'Hikvision pour avoir les infos sur leur caméras
Besoin d'un ordinateur portable pour le groupe	L'utilisation de la caméra demandait beaucoup de temps de dev et un pc pour expérimenter	Mathis à récupéré son ordinateur portable pour ne pas à avoir à interférer avec le travail de Maxence qui avait aussi besoin du sient.

6.2. Analyse détaillée de l'incident majeur : conflit de port WebSocket 8585

Lors des phases de tests, le serveur affichait régulièrement l'erreur critique suivante :
`[LOG] ERREUR SERVEUR WS : [Errno 10048] error while attempting to bind on address ('127.0.0.1', 8585)`

La commande Windows habituelle `taskkill /F /IM python.exe` répondait qu'aucun processus n'était trouvé, car l'interpréteur Python du Microsoft Store était exécuté sous le nom de processus `python3.12.exe` . L'analyse a été résolue méthodiquement via PowerShell :

1 - Identification précise du processus occupant le port 8585

```
Get-Process -Id (Get-NetTCPConnection -LocalPort 8585).OwningProcess
```

Résultat obtenu : `ProcessName = python3.12, Id = 2668`

2 - Destruction forcée du processus

```
Stop-Process -Id 2668 -Force
```

Afin de prévenir cet incident, le script server_ipcam.py a été enrichi d'un message d'aide explicite guidant l'utilisateur vers cette commande en cas d'erreur au démarrage. Ont ne peut pas vraiment prévoir ce genre de problème, il faut tester le code pour détecter les erreurs.
Ce problème à été résolu grâce à l'aide d'une IA.

7. Conclusion et perspectives

7.1. Bilan du travail réalisé

L'ensemble des objectifs assignés à la Tâche 1 du Mini-Projet 1 a été atteint avec succès . La chaîne globale de vidéosurveillance comprenant l'alimentation électrique de la caméra IP Hikvision, la liaison réseau sécurisée en TCP, le traitement matriciel OpenCV, la diffusion WebSocket temps réel et l'IHM web chartée est entièrement fonctionnelle et validée par une série de tests rigoureux .

7.2. Perspectives d'évolution

Plusieurs pistes d'amélioration technique peuvent être envisagées pour faire évoluer le système:

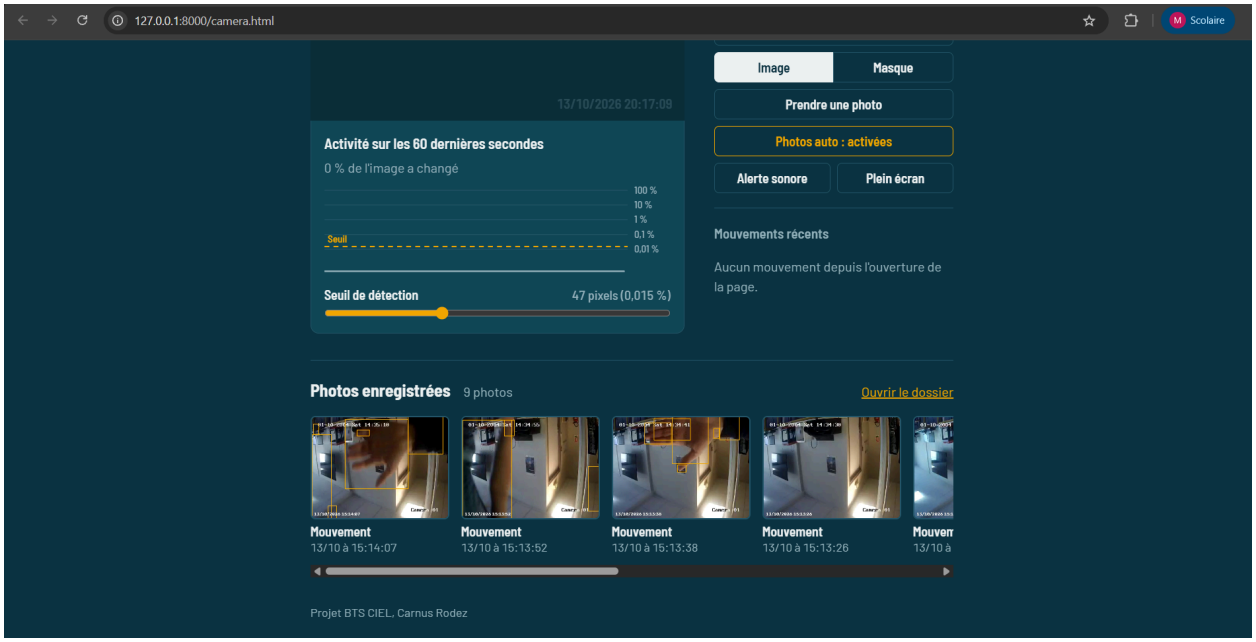
- **Intégration d'un modèle de reconnaissance humaine :** Remplacer la méthode de détection actuelle par une plus avancée (voir même une IA) afin d'effectuer une classification d'objets (distinction entre une personne, un animal ou un véhicule) et éliminer totalement les fausses alertes .
- **Envoi de notifications distantes :** Implémenter un service de notification (via Telegram, SMS ou e-mail) lors du déclenchement d'un événement en période d'occupation non autorisée.

8. Annexes techniques

8.1. Interface web



Document 1 - Interface web de la vidéosurveillance (capture ½)



Document 2 - Interface web de la vidéosurveillance (capture 2/2)